Document number: P0575R1
Date: 2017-03-03
Project: Programming Language C++
Reference: ISO/IEC IS 14882:2014
Reply to: William M. Miller
Edison Design Group, Inc.
wmm@edg.com

Core Language Working Group "ready" Issues for the February, 2017 (Kona) meeting

History:

Revision 1: Issue 1622 was removed because its resolution was modified since the pre-meeting mailing. Issue 2191 was removed because its resolution was already applied to the working paper as part of paper P0003R5.

Section references in this document reflect the section numbering of document WG21 N4606.

---

426. Identically-named variables, one internally and one externally linked, allowed?

Section: 3.5 [basic.link] **Status:** ready **Submitter:** Steve Adamczyk **Date:** 2 July 2003

An example in 3.5 [basic.link] paragraph 6 creates two file-scope variables with the same name, one with internal linkage and one with external.

```
static void f();
static int i = 0;                       //1
void g() {
        extern void f();                // internal linkage
        int i;                          //2: i has no linkage
        {
                extern void f();        // internal linkage
                extern int i;           //3: external linkage
        }
}
```

Is this really what we want? C99 has 6.2.2.7/7, which gives undefined behavior for having an identifier appear with internal and external linkage in the same translation unit. C++ doesn't seem to have an equivalent.

Notes from October 2003 meeting:

We agree that this is an error. We propose to leave the example but change the comment to indicate that line //3 has undefined behavior, and elsewhere add a normative rule giving such a case undefined behavior.

Proposed resolution (October, 2005) [SUPERSEDED]:

Change 3.5 [basic.link] paragraph 6 as indicated:

> ...Otherwise, if no matching entity is found, the block scope entity receives external linkage. **If, within a translation unit, the same entity is declared with both internal and external linkage, the behavior is undefined.**
>
> [*Example:*
>
> ```
> static void f();
> static int i = 0;           // 1
> void g () {
>     extern void f ();       // internal linkage
>     int i;                  // 2: i has no linkage
>     {
>         extern void f ();   // internal linkage
>         extern int i;       // 3: external linkage
>     }
> }
> ```

~~There are three objects named `i` in this program. The object with internal linkage introduced by the declaration in global scope (line `//1`), the object with automatic storage duration and no linkage introduced by the declaration on line `//2`, and the object with static storage duration and external linkage introduced by the declaration on line `//3`.~~ **Without the declaration at line `//2`, the declaration at line `//3` would link with the declaration at line `//1`. But because the declaration with internal linkage is hidden, `//3` is given external linkage, resulting in a linkage conflict.** *—end example*]

Notes from the April 2006 meeting:

According to 3.5 [basic.link] paragraph 9, the two variables with linkage in the proposed example are not "the same entity" because they do not have the same linkage. Some other formulation will be needed to describe the relationship between those two variables.

Notes from the October 2006 meeting:

The CWG decided that it would be better to make a program with this kind of linkage mismatch ill-formed instead of having undefined behavior.

Proposed resolution (November, 2016):

Change 3.5 [basic.link] paragraph 6 as follows:

...Otherwise, if no matching entity is found, the block scope entity receives external linkage. **If, within a translation unit, the same entity is declared with both internal and external linkage, the program is ill-formed.** [*Example:*

```
static void f();
static int i = 0;      // #1
void g() {
  extern void f();     // internal linkage
  int i;               // #2: i has no linkage
  {
    extern void f();   // internal linkage
    extern int i;      // #3 external linkage, ill-formed
  }
}
```

~~There are three objects named `i` in this program. The object with internal linkage introduced by the declaration in global scope (line #1), the object with automatic storage duration and no linkage introduced by the declaration on line #2, and the object with static storage duration and external linkage introduced by the declaration on line #3.~~ **Without the declaration at line #2, the declaration at line #3 would link with the declaration at line #1. Because the declaration with internal linkage is hidden, however, #3 is given external linkage, making the program ill-formed.** *—end example*]

---

727. In-class explicit specializations

Section: 14.7.3 [temp.expl.spec] **Status:** ready **Submitter:** Faisal Vali **Date:** 5 October, 2008

14.7.3 [temp.expl.spec] paragraph 2 requires that explicit specializations of member templates be declared in namespace scope, not in the class definition. This restriction does not apply to partial specializations of member templates; that is,

```
struct A {
  template<class T> struct B;
  template <class T> struct B<T*> { }; // well-formed
  template <> struct B<int*> { }; // ill-formed
};
```

There does not seem to be a good reason for this inconsistency.

Additional note (October, 2013):

EWG has requested CWG to consider resolving this issue. See EWG issue 41.

Additional note, November, 2014:

See also paper N4090.

Proposed resolution (March, 2017):

1. Change 14.5.5 [temp.class.spec] paragraph 5 as follows:

   A class template partial specialization may be declared ~~or redeclared~~ in any ~~namespace~~ scope in which the corresponding primary template may be defined (7.3.1.2 [namespace.memdef] ~~and~~ , 9.2 [class.mem], 14.5.2 [temp.mem]). [*Example:*

```
template<class T> struct A {
  struct C {
    template<class T2> struct B { };
    template<class T2> struct B<T2**> { };  // partial specialization #1
  };
};

// partial specialization of A<T>::C::B<T2>
template<class T> template<class T2>
  struct A<T>::C::B<T2*> { };    // #2


A<short>::C::B<int*> absip; // uses partial specialization #2
```

   —*end example*]

2. Change 14.7.3 [temp.expl.spec] paragraph 2 as follows:

   An explicit specialization ~~shall be declared in a namespace enclosing the specialized template. An explicit specialization whose *declarator-id* or *class-head-name* is not qualified shall be declared in the nearest enclosing namespace of the template, or, if the namespace is inline (7.3.1 [namespace.def]), any namespace from its enclosing namespace set. Such a declaration may also be a definition~~ **may be declared in any scope in which the corresponding primary template may be defined (7.3.1.2 [namespace.memdef], 9.2 [class.mem], 14.5.2 [temp.mem])**. ~~If the declaration is not a definition, the specialization may be defined later (7.3.1.2 [namespace.memdef]).~~

1677. Constant initialization via aggregate initialization

Section: 3.6.2 [basic.start.static] **Status:** ready **Submitter:** Daniel Krügler **Date:** 2013-05-05

The resolution of issue 1489 added wording regarding value initialization to 3.6.2 [basic.start.static] paragraph 2 in an attempt to clarify the status of an example like

```
int a[1000]{};
```

However, this example is aggregate initialization, not value initialization. Also, now that we allow *brace-or-equal-initializer*s in aggregates, this wording also needs to be updated to allow an aggregate with constant non-static data member initializers to qualify for constant initialization.

Proposed resolution (November, 2017):

1. Change 3.6.2 [basic.start.static] paragraph 2 as follows, converting the bulleted list into running text as indicated:

   A *constant initializer* for ~~an~~ **a variable or temporary** object `o` is an ~~expression that~~ **initializer whose full-expression** is a constant expression, except that ~~it~~ **if `o` is an object, such an initializer** may also invoke `constexpr` constructors for `o` and its subobjects even if those objects are of non-literal class types. [*Note:* Such a class may have a non-trivial destructor —*end note*] *Constant initialization* is performed~~:~~

   - ~~if each full-expression (including implicit conversions) that appears in the initializer of a reference with static or thread storage duration is a constant expression (5.20 [expr.const]) and the reference is bound to a glvalue designating an object with static storage duration, to a temporary object (see 12.2 [class.temporary]) or subobject thereof, or to a function;~~

   - if ~~an~~ **a variable or temporary** object with static or thread storage duration is initialized by a ~~constructor call, and if the initialization full-expression is a~~ constant initializer for the ~~object;~~ **entity**

- ~~if an object with static or thread storage duration is not initialized by a constructor call and if either the object is value-initialized or every full-expression that appears in its initializer is a constant expression~~.

If constant initialization is not performed...

2. Change 5.20 [expr.const] paragraph 2 as follows:

~~*A conditional-expression*~~ **An expression** `e` is a *core constant expression* unless...

1710. Missing `template` keyword in *class-or-decltype*

Section: 10 [class.derived] **Status:** ready **Submitter:** Richard Smith **Date:** 2013-07-03

A *class-or-decltype* is used as a *base-specifier* and as a *mem-initializer-id* that names a base class. It is specified in 10 [class.derived] paragraph 1 as:

class-or-decltype:
nested-name-specifier$_{opt}$ *class-name*
decltype-specifier

Consequently, a declaration like

```
template<typename T> struct D : T::template B<int>::template C<int> {};
```

is ill-formed, although most implementations accept it; some actually require the use of the `template` keyword, although the relevant wording in 14.2 [temp.names] paragraph 4 only requires it in a *qualified-id*, not in a *class-or-decltype*. It would probably be good to add a production like

nested-name-specifier `template` *simple-template-id*

to the definition of *class-or-decltype* and explicitly mention those contexts in 14.2 [temp.names] as not requiring use of the `template` keyword.

Additional note (January, 2014):

This is effectively issues 314 and 343.

See also issue 1812.

Proposed resolution (February, 2014) [SUPERSEDED]:

1. Change 9 [class] paragraph 3 as follows:

If a class is marked with the *class-virt-specifier* `final` and it appears as a ~~*base-type-specifier*~~ ***class-or-decltype*** in a *base-clause* (Clause 10 [class.derived]), the program is ill-formed. Whenever a *class-key* is followed...

2. Change the grammar in 10 [class.derived] paragraph 1 as follows:

base-specifier:
attribute-specifier-seq$_{opt}$ ~~*base-type-specifier*~~ ***class-or-decltype***
attribute-specifier-seq$_{opt}$ `virtual` *access-specifier*$_{opt}$ ~~*base-type-specifier*~~ ***class-or-decltype***
attribute-specifier-seq$_{opt}$ *access-specifier* `virtual`$_{opt}$ ~~*base-type-specifier*~~ ***class-or-decltype***
class-or-decltype:
nested-name-specifier$_{opt}$ *class-name*
nested-name-specifier **`template`** ***simple-template-id***
decltype-specifier
~~*base-type-specifier:*~~
~~*class-or-decltype*~~
access-specifier:
...

3. Delete paragraph 4 and change paragraph 5 of 14.2 [temp.names] as follows, splitting paragraph 5 into two paragraphs and moving the example from paragraph 4 into paragraph 5:

~~When the name of a member template specialization appears after `.` or `->` in a *postfix-expression* or after a *nested-name-specifier* in a *qualified-id*, and the object expression of the *postfix-expression* is type-dependent or the *nested-name-specifier* in the *qualified-id* refers to a dependent type, but the name is not a member of the current instantiation (14.6.2.1 [temp.dep.type]), the member template name must be prefixed by the keyword `template`. Otherwise the name is assumed to name a non-template. [*Example: ... —end example*]~~

A name prefixed by the keyword `template` shall be a *template-id* or the name shall refer to a class template **or alias template**. [*Note:* The keyword `template` may not be applied to non-template members of class templates. *—end note*] **The *nested-name-specifier* (_N4567_.5.1.1 [expr.prim.general]) of**

- **a *class-head-name* (Clause 9 [class]) or *enum-head* (7.2 [dcl.enum]) (if any) or**
- **a *qualified-id* in a *declarator-id* (Clause 8 [dcl.decl]),**

or a *nested-name-specifier* directly contained in such a *nested-name-specifier* (recursively), shall not be of the form

***nested-name-specifier* `template` *simple-template-id* `::`**

[*Note:* That is, a *simple-template-id* shall not be prefixed by the keyword `template` in these cases. *—end note*]

The keyword `template` is optional in a *typename-specifier* (14.6 [temp.res]), *elaborated-type-specifier* (7.1.7.3 [dcl.type.elab]), *using-declaration* (7.3.3 [namespace.udecl]), or *class-or-decltype* (Clause 10 [class.derived]), and in recursively directly-contained *nested-name-specifier*s thereof. In these contexts, a `<` token is always assumed to introduce a *template-argument-list*. [*Note:* Thus, if the preceding name is not a *template-name*, the program is ill-formed. *—end note*] In other contexts, when the name of a member template specialization appears after a *nested-name-specifier* that denotes a dependent type, but the name is not a member of the current instantiation, the member template name shall be prefixed by the keyword `template`. Similarly, when the name of a member template specialization appears after `.` or `->` in a *postfix-expression* (5.2 [expr.post]) and the object expression of the *postfix-expression* is type-dependent, but the name is not a member of the current instantiation (14.6.2.1 [temp.dep.type]), the member template name shall be prefixed by the keyword `template`. Otherwise, the name is assumed to name a non-template. [*Example:*

```
<From original paragraph 4>
```

—end example] [*Note:* As is the case with the `typename` prefix...

This resolution also resolves issues 314, 343, 1794, and 1812.

Additional note, November, 2014:

Concerns have been expressed over the clarity and organization of the proposed resolution, so the issue has been moved back to "review" status to allow CWG to address these concerns.

Proposed resolution, March, 2017:

1. Change 9 [class] paragraph 3 as follows:

   If a class is marked with the *class-virt-specifier* `final` and it appears as a ~~*base-type-specifier*~~ ***class-or-decltype*** in a *base-clause* (Clause 10 [class.derived]), the program is ill-formed. Whenever a *class-key* is followed by a *class-head-name*...

2. Change the grammar in 10 [class.derived] paragraph 1 as follows:

   base-specifier:
   attribute-specifier-seq$_{opt}$ ~~*base-type-specifier*~~ ***class-or-decltype***
   attribute-specifier-seq$_{opt}$ `virtual` *access-specifier*$_{opt}$ ~~*base-type-specifier*~~ ***class-or-decltype***
   attribute-specifier-seq$_{opt}$ *access-specifier* `virtual`$_{opt}$ ~~*base-type-specifier*~~ ***class-or-decltype***

   class-or-decltype:
   nested-name-specifier$_{opt}$ *class-name*
   nested-name-specifier* `template` *simple-template-id
   decltype-specifier

~~*base-type-specifier:*~~
~~*class-or-decltype*~~

3. Change 10 [class.derived] paragraph 2 as followx:

> ~~The type denoted by a *base-type-specifier*~~ **A *class-or-decltype*** shall ~~be~~ **denote** a class type that is not an incompletely defined class (Clause 9 [class])~~; this~~**. The** class **denoted by the *class-or-decltype* of a *base-specifier*** is called a *direct base class* for the class being defined. During the lookup for a base class name...

4. Change 14.2 [temp.names] paragraphs 4 and 5 as follows:

> ~~When the name of a member template specialization appears after . or -> in a *postfix-expression* or after a *nested-name-specifier* in a *qualified-id*, and the object expression of the *postfix-expression* is type-dependent or the *nested-name-specifier* in the *qualified-id* refers to a dependent type, but the name is not a member of the current instantiation~~ **The keyword `template` is said to appear at the top level in a *qualified-id* if it appears outside of a *template-argument-list* or *decltype-specifier*. In a *qualified-id* of a *declarator-id* or in a *qualified-id* formed by a *class-head-name* (Clause 9 [class]) or *enum-head-name* (7.2 [dcl.enum]), the keyword `template` shall not appear at the top level. In a *qualified-id* used as the name in a *typename-specifier* (14.6 [temp.res]), *elaborated-type-specifier* (7.1.7.3 [dcl.type.elab]), *using-declaration* (7.3.3 [namespace.udecl]), or *class-or-decltype* (Clause 10 [class.derived]), an optional keyword `template` appearing at the top level is ignored. In these contexts, a < token is always assumed to introduce a *template-argument-list*. In all other contexts, when naming a template specialization of a member of an unknown specialization** (14.6.2.1 [temp.dep.type]), the member template name ~~must~~ **shall** be prefixed by the keyword `template`. ~~Otherwise the name is assumed to name a non-template.~~ [*Example:*...
>
> A name prefixed by the keyword `template` shall be a *template-id* or the name shall refer to a class template **or an alias template**. [*Note:* The keyword `template` may not be applied to non-template members of class templates. *—end note*]...

5. Change 14.6 [temp.res] paragraph 5 as follows:

> A qualified name used as the name in a ~~*mem-initializer-id*, a *base-specifier*,~~ ***class-or-decltype* (Clause 10 [class.derived])** or an *elaborated-type-specifier* is implicitly assumed to name a type, without the use of the `typename` keyword. In a *nested-name-specifier* that immediately contains a *nested-name-specifier* that depends on a template parameter, the identifier or *simple-template-id* is implicitly assumed to name a type, without the use of the `typename` keyword. [*Note:* The `typename` keyword is not permitted by the syntax of these constructs. *—end note*]

1860. What is a “direct member?”

Section: 9.3 [class.union] **Status:** ready **Submitter:** Dawn Perchik **Date:** 2014-02-13

The term “direct member” is used in 9.3 [class.union] paragraph 8 but is not defined. It might be better to refer to the class's *member-specification* instead.

Additional note, October, 2015:

This issue is expected to be addressed by the wording of N4532 or a successor thereof (“Default Comparisons”).

Proposed resolution (November, 2016):

1. Change 8.6.1 [dcl.init.aggr] paragraph 2 as follows:

> The *elements* of an aggregate are:
>
> - for an array, the array elements in increasing subscript order, or
> - for a class, the direct base classes in declaration order followed by the direct **non-static data** members **that are not members of an anonymous union,** in declaration order.

2. Change 9.2 [class.mem] paragraph 1 as follows:

> The *member-specification* in a class definition declares the full set of members of the class; no member can be added elsewhere. **A *direct member* of a class `X` is a member of `X` that was first declared within the *member-specification* of `X`, including anonymous union objects and direct members thereof.** Members of a class are...

3. Change 9.3.1 [class.union.anon] paragraph 1 as follows:

> A union of the form
>
> `union {` *member-specification* `} ;`
>
> is called an *anonymous union*; it defines an unnamed **type and an unnamed** object of ~~unnamed~~ **that** type **called an *anonymous union object***. Each *member-declaration* in the *member-specification* of an anonymous union
>
> ...

2174. Unclear rules for friend definitions in templates

Section: 14.5.4 [temp.friend] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-09-17

According to 14.5.4 [temp.friend] paragraph 4,

> When a function is defined in a friend function declaration in a class template, the function is instantiated when the function is odr-used (3.2 [basic.def.odr]). The same restrictions on multiple declarations and definitions that apply to non-template function declarations and definitions also apply to these implicit definitions.

This seems to imply that:

1. Instantiating a class template that contains a friend function definition instantiates the declaration, but not the definition, of that friend function, as usual (but see below).
2. If the function is odr-used, a definition is instantiated for each such class template specialization whose template had a definition.
3. If that results in multiple definitions, the program is ill-formed as usual.

The intent appears to be that the instantiated friend function declarations should be treated as if they were definitions, but that's not clear from the wording. This wording is also missing similar provisions for friend function template definitions; there is implementation divergence on the treatment of such cases.

There also does not appear to be wording that says that instantiating a class template specialization results in the instantiation of friend functions declared/defined therein (the relevant wording was removed from this section by issue 329). Presumably this should be covered in 14.7.1 [temp.inst] paragraph 1, which also includes the following wording that could be reused for the `friend` case:

> However, for the purpose of determining whether an instantiated redeclaration of a member is valid according to 9.2 [class.mem], a declaration that corresponds to a definition in the template is considered to be a definition.

Also, the reliance on odr-use to trigger friend instantiation is out of date, as there are other contexts that can require an instantiation when there is no odr-use (a `constexpr` function invoked within an unevaluated operand).

Proposed resolution (October, 2015) [SUPERSEDED]:

1. Delete 14.5.4 [temp.friend] paragraph 4:

> ~~When a function is defined in a friend function declaration in a class template, the function is instantiated when the function is odr-used (3.2 [basic.def.odr]). The same restrictions on multiple declarations and definitions that apply to non-template function declarations and definitions also apply to these implicit definitions.~~

2. Change 14.7.1 [temp.inst] paragraph 1 as follows:

> ...The implicit instantiation of a class template specialization causes the implicit instantiation of the declarations, but not of the definitions, default arguments, or *exception-specification*s of the class member functions, member classes, scoped member enumerations, static data members**,** ~~and~~ member templates**, and friends**; and it causes the implicit instantiation of the definitions of unscoped member enumerations and member anonymous unions. However, for the purpose of determining whether an instantiated redeclaration ~~of a member~~ is valid according to **3.2 [basic.def.odr] and** 9.2 [class.mem], a declaration that corresponds to a definition in the template is considered to be a definition. [*Example:*...

3. Change 14.7.1 [temp.inst] paragraph 3 as follows:

Unless a function template specialization has been explicitly instantiated or explicitly specialized, the function template specialization is implicitly instantiated when the specialization is referenced in a context that requires a function definition to exist. **A function whose declaration was instantiated from a friend function definition is implicitly instantiated when it is referenced in a context that requires a function definition to exist.** Unless a call is to a function template explicit specialization or to a member function of an explicitly specialized class template, a default argument for a function template or a member function of a class template is implicitly instantiated when the function is called in a context that requires the value of the default argument.

Proposed resolution (November, 2016):

1. Delete 14.5.4 [temp.friend] paragraph 4:

   ~~When a function is defined in a friend function declaration in a class template, the function is instantiated when the function is odr-used (3.2 [basic.def.odr]). The same restrictions on multiple declarations and definitions that apply to non-template function declarations and definitions also apply to these implicit definitions.~~

2. Change 14.7.1 [temp.inst] paragraph 1 as follows, splitting it into two paragraphs as indicated:

   ... [*Note:* Within a template declaration, a local class (9.4 [class.local]) or enumeration and the members of a local class are never considered to be entities that can be separately instantiated (this includes their default arguments, *exception-specification*s, and non-static data member initializers, if any). As a result, the dependent names are looked up, the semantic constraints are checked, and any templates used are instantiated as part of the instantiation of the entity within which the local class or enumeration is declared. —*end note*]

   The implicit instantiation of a class template specialization causes the implicit instantiation of the declarations, but not of the definitions, default arguments, or *exception-specification*s of the class member functions, member classes, scoped member enumerations, static data members, ~~and~~ member templates**, and friends**; and it causes the implicit instantiation of the definitions of unscoped member enumerations and member anonymous unions. However, for the purpose of determining whether an instantiated redeclaration ~~of a member~~ is valid according to **3.2 [basic.def.odr] and** 9.2 [class.mem], a declaration that corresponds to a definition in the template is considered to be a definition. [*Example:*

```
template<class T, class U>
struct Outer {
  template<class X, class Y> struct Inner;
  template<class Y> struct Inner<T, Y>;     // #1a
  template<class Y> struct Inner<T, Y> { }; // #1b; OK: valid redeclaration of #1a
  template<class Y> struct Inner<U, Y> { }; // #2
};

Outer<int, int> outer; // error at #2
```

`Outer<int, int>::Inner<int, Y>` is redeclared at #1b. (It is not defined but noted as being associated with a definition in `Outer<T, U>`.) #2 is also a redeclaration of #1a. It is noted as associated with a definition, so it is an invalid redeclaration of the same partial specialization.

```
template<typename T> struct Friendly {
  template<typename U> friend int f(U) { return sizeof(T); }
};
Friendly<char> fc;
Friendly<float> ff; // ill-formed: produces second definition of f(U)
```

—*end example*]

3. Change 14.7.1 [temp.inst] paragraph 3 as follows:

   Unless a function template specialization has been explicitly instantiated or explicitly specialized, the function template specialization is implicitly instantiated when the specialization is referenced in a context that requires a function definition to exist. **A function whose declaration was instantiated from a friend function definition is implicitly instantiated when it is referenced in a context that requires a function definition to exist.** Unless a call is to a function template explicit specialization or to a member function of an explicitly specialized class template, a default argument for a function template or a member function of a class template is implicitly instantiated when the function is called in a context that requires the value of the default argument.

2196. Zero-initialization with virtual base classes

Section: 8.6 [dcl.init] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-11-06

[Detailed description pending.]

Proposed resolution (November, 2016):

Change 8.6 [dcl.init] bullet 6.2 as follows:

> To *zero-initialize* an object or reference of type `T` means:
>
> - if `T` is a scalar type (3.9 [basic.types]), the object is initialized to the value obtained by converting the integer literal `0` (zero) to `T`;[103]
> - if `T` is a (possibly cv-qualified) non-union class type, each non-static data member**, each non-virtual base class subobject,** and**, if the object is not a base class subobject,** each **virtual** base- class subobject is zero-initialized and padding is initialized to zero bits;
> - ...

2198. Linkage of enumerators

Section: 3.5 [basic.link] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-11-12

According to the rules in 3.5 [basic.link] paragraph 4, the enumerators of an enumeration type with linkage also have linkage. Having same-named enumerators in different translation units would seem to be innocuous. Is there a rationale for this rule?

Proposed resolution (March, 2017):

1. Delete 3.5 [basic.link] bullet 4.5:

   > An unnamed namespace or a namespace declared directly or indirectly within an unnamed namespace has internal linkage. All other namespaces have external linkage. A name having namespace scope that has not been given internal linkage above has the same linkage as the enclosing namespace if it is the name of
   >
   > - ...
   > - ~~an enumerator belonging to an enumeration with linkage; or~~
   > - ...

2. Change 3.5 [basic.link] paragraph 9 as follows:

   > Two names that are the same (Clause 3 [basic]) and that are declared in different scopes shall denote the same variable, function, type, ~~enumerator,~~ template or namespace if...

2205. Restrictions on use of `alignas`

Section: 7.6.1 [dcl.attr.grammar] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-11-30

According to 7.6.1 [dcl.attr.grammar] paragraph 5, a program is ill-formed if an *attribute* appertains to an entity or statement to which it is not allowed to apply. Presumably an *alignment-specifier* should have the same restriction.

Proposed resolution (November, 2016):

Change 7.6.1 [dcl.attr.grammar] paragraph 5 as follows:

> Each *attribute-specifier-seq* is said to *appertain* to some entity or statement, identified by the syntactic context where it appears (Clause 6 [stmt.stmt], Clause 7 [dcl.dcl], Clause 8 [dcl.decl]). If an *attribute-specifier-seq* that appertains to some entity or statement contains an *attribute* **or *alignment-specifier*** that is not allowed to apply to that entity or statement, the program is ill-formed. If an *attribute-specifier-seq* appertains to a friend declaration (11.3 [class.friend]),

that declaration shall be a definition. No *attribute-specifier-seq* shall appertain to an explicit instantiation (14.7.2 [temp.explicit]).

2218. Ambiguity and namespace aliases

Section: 3.4 [basic.lookup] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-12-29

There is implementation divergence on the status of the following example:

```
namespace A { namespace B { int x; } }
namespace C { namespace B = A::B; }
using namespace A;
using namespace C;
int x = B::x;
```

This should presumably be valid: the lookup of `B` finds `A::B` and `C::B`, but it is not ambiguous because they denote the same entity. A similar example with a *using-declaration* or *alias-declaration* seems to be universally accepted. Perhaps the lookup rules need to be clarified regarding the status of this example.

Proposed resolution (November, 2016):

Change 3.4 [basic.lookup] paragraph 1 as follows:

> The name lookup rules apply uniformly to all names (including *typedef-name*s (7.1.3 [dcl.typedef]), *namespace-name*s (7.3 [basic.namespace]), and *class-name*s (9.1 [class.name])) wherever the grammar allows such names in the context discussed by a particular rule. Name lookup associates the use of a name with a ~~declaration~~ **set of declarations** (3.1 [basic.def]) of that name. ~~Name lookup shall find an unambiguous declaration for the name (see 10.2 [class.member.lookup]). Name lookup may associate more than one declaration with a name if it finds the name to be a function name;~~ **The declarations found by name lookup shall either all declare the same entity or shall all declare functions; in the latter case,** the declarations are said to form a set of overloaded functions (13.1 [over.load]). Overload resolution...

2248. Problems with sized delete

Section: 5.3.5 [expr.delete] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-03-14

[Detailed description pending.]

Proposed resolution (December, 2016):

Change 5.3.5 [expr.delete] paragraph 11 as follows:

> When a *delete-expression* is executed, the selected deallocation function shall be called with the address of the ~~block of storage to be reclaimed~~ **most-derived object in the *delete object* case, or the address of the object suitably adjusted for the array allocation overhead (5.3.4 [expr.new]) in the *delete array* case,** as its first argument. If a deallocation function with a parameter of type `std::align_val_t` is used, the alignment of the type of the object to be deleted is passed as the corresponding argument. If a deallocation function with a parameter of type `std::size_t` is used, the size of the ~~block~~ **most-derived type, or of the array plus allocation overhead, respectively,** is passed as the corresponding argument. **[*Note:* If this results in a call to a usual deallocation function, and either the first argument was not the result of a prior call to a usual allocation function or the second argument was not the corresponding argument in said call, the behavior is undefined (18.6.2.1 [new.delete.single], 18.6.2.2 [new.delete.array]). —*end note*]**

2251. Unreachable enumeration list-initialization

Section: 8.6.4 [dcl.init.list] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-03-22

[Detailed description pending.]

Proposed resolution (December, 2016):

Reorder the bullets in 8.6.4 [dcl.init.list] paragraph 3 as follows:

> List-initialization of an object or reference of type `T` is defined as follows:
>
> - ...
> - Otherwise, if `T` is a class type, constructors are considered...
> - **Otherwise, if `T` is an enumeration with a fixed underlying type (7.2 [dcl.enum]), the *initializer-list* has a single element `v`, and the initialization is direct-list-initialization, the object is initialized with the value `T(v)` (5.2.3 [expr.type.conv]); if a narrowing conversion is required to convert `v` to the underlying type of `T`, the program is ill-formed. [*Example:*...**
> - Otherwise, if the initializer list has a single element of type `E`...
> - Otherwise, if `T` is a reference type...
> - ~~Otherwise, if `T` is an enumeration with a fixed underlying type...~~